

Experiment 7: Scalable Read APIs with Caching & Optimization - Now Secure APIs with JWT, RBAC & Encryption

This document explains the advanced backend security concepts implemented in the Secure Auth Portal project, directly following the requirements of Experiment 7. It also provides **real-world production examples** to justify why these techniques are essential in industry-standard applications.

1. Spring Security Architecture

Why it's needed: Securing APIs manually is prone to errors, missing edge cases, and vulnerabilities. Spring Security provides a robust, layered security architecture to handle authentication (who you are) and authorization (what you can do).

How it's implemented:

- We configure a `SecurityFilterChain` that intercepts every HTTP request.
- We define public endpoints (e.g., `/api/auth/login`) and protected endpoints.
- We implement `UserDetailsService` to fetch user credentials and roles from the database.

 **Production Example (Banking Systems):** When you open a banking app, the request doesn't directly hit the API that transfers money. It passes through a complex Security Filter Chain that checks if the request is authenticated, if it's not a CSRF attack, and if your session is valid, ensuring multi-layered protection.

2. JWT Authentication (JSON Web Tokens)

Why it's needed: Traditional sessions require the server to store a session ID for every logged-in user in memory. In a cloud environment with multiple servers (horizontal scaling), storing session state is difficult and inefficient.

How it's implemented:

- We created a `JwtAuthFilter` that extracts the token from the `Authorization: Bearer <token>` header.
- The backend generates a JWT containing the user's details and signs it with a secret key upon successful login.
- Subsequent requests are validated purely mathematically using the signature, requiring no database lookup for session validation.

 **Production Example (Microservices):** Companies like Netflix use microservices. A user authenticates once with an Auth Service, gets a JWT, and then passes that JWT to the Video Service, Billing Service, etc. None of these downstream services need to contact the database to verify the user—they just cryptographically verify the JWT signature.

3. Role-Based Access Control (RBAC)

Why it's needed: Not all authenticated users should have the same permissions. A regular user shouldn't be able to delete accounts or view administrative data. This follows the **Principle of Least Privilege**.

How it's implemented:

- We assign roles (e.g., `USER` , `ADMIN`) to users in the database.
- We secure specific Controller methods using `@PreAuthorize("hasRole('ADMIN')")` .
- If a `USER` tries to access an `ADMIN` endpoint, Spring Security automatically throws a 403 Forbidden error.

 **Production Example (SaaS Platforms):** In a platform like GitHub or Slack, there are Owners, Admins, and Members. The backend code uses RBAC to ensure that only an 'Owner' role can delete a workspace, while a 'Member' role can only read and write messages.

4. Access & Refresh Token Strategy (Token Rotation)

Why it's needed: If a JWT never expires, it's a massive security risk if stolen. If it expires too quickly (e.g., 15 minutes), the user will be constantly logged out, ruining the User Experience (UX).

How it's implemented:

- We issue two tokens: a short-lived **Access Token** (e.g., 15 mins) and a long-lived **Refresh Token** (e.g., 7 days).
- The frontend uses the Access Token for API calls. When it expires, it silently sends the Refresh Token to a special `/refresh` endpoint to get a new Access Token without the user having to type their password again.

 **Production Example (Mobile Apps):** Think of the Instagram app. You log in once and seemingly never log out. Behind the scenes, your short-lived access token is constantly expiring, but the app uses a securely stored refresh token to get a new one automatically. If your phone is stolen, Instagram can revoke the refresh token remotely, blocking access.

5. AES Encryption & Secure Credential Storage

Why it's needed: Storing sensitive data like Third-Party OAuth tokens (Google, Facebook) or personal identifiable information (PII) in plaintext in the database is disastrous if a data breach occurs.

How it's implemented:

- We use the **AES** (Advanced Encryption Standard) symmetric encryption algorithm.
- Sensitive data is encrypted before being saved to the database.
- The data is only decrypted in memory when actively needed by the application.

 **Production Example (Password Managers / Healthcare):** In applications like 1Password or hospital management systems (HIPAA compliant), databases are assumed to be a vulnerable layer. Every sensitive field is encrypted using strong keys (like AES-256). Even if a hacker steals the entire database dump, the data remains unreadable gibberish without the encryption keys.

6. Architecture & Folder Structure

To implement these security features cleanly, the backend is organized into a modular layered architecture, and the frontend is structured to securely manage state.

Backend (`/backend`)

```
backend/  
├── pom.xml                # Maven configuration (Spring Security, JWT, JPA, H2)
```

```

└─ src/main/
  └─ resources/
    └─ application.properties # Database & JWT Configurations
  └─ java/com/example/backend/
    └─ BackendApplication.java # Application entry point
    └─ config/
      └─ SecurityConfig.java # Spring Security Filter Chain & Rules
    └─ controller/
      └─ AuthController.java # Endpoints for Login, Register, and RBAC tests
    └─ entity/
      └─ User.java # JPA Entity for the users table
    └─ repository/
      └─ UserRepository.java # DB access for user validation
    └─ security/
      └─ JwtAuthFilter.java # Intercepts requests to validate tokens
      └─ JwtUtils.java # Utility for generating/parsing JWTs
    └─ util/
      └─ AesUtil.java # AES-256 Encryption/Decryption utility

```

Frontend (/frontend)

```

frontend/
└─ package.json # Node.js dependencies (React, Vite, Axios)
└─ vite.config.js # Vite build configuration
└─ src/
  └─ App.jsx # Main React component (Routing & State)
  └─ App.css # Premium CSS styling (glassmorphism, alerts)
  └─ index.css # Global CSS resets
  └─ components/ # Reusable UI components
  └─ pages/ # Views (Login, Dashboard, Admin)
  └─ context/ # React Context for Auth State

```

7. Algorithm: Secure Authentication Flow

The following algorithm outlines how the system processes a secure login and handles subsequent protected requests:

1. User Login Request:

- The user submits their `username` and `password` via the React frontend.

2. Credential Verification:

- The `AuthController` receives the request and asks the `AuthenticationManager` to verify the credentials against the database.

3. Token Generation:

- If valid, `JwtUtils` generates a JWT containing the user's identity and `Role`.
- The backend responds with the JWT.

4. Token Storage:

- The frontend stores the JWT in memory (or a secure HttpOnly cookie).

5. Subsequent API Request:

- The frontend attaches the token in the `Authorization: Bearer <token>` header for any protected API call.

6. Request Interception (Filter Chain):

- `JwtAuthFilter` intercepts the incoming request.
- It extracts the token, verifies the cryptographic signature, and checks the expiration date.

7. Role Authorization (RBAC):

- If the token is valid, Spring Security checks if the user's `Role` matches the `@PreAuthorize` requirements of the requested endpoint.

8. Response:

- If authorized, the backend processes the request (decrypting sensitive data if necessary using `AesUtil`) and returns the secure data.
- If unauthorized, a `403 Forbidden` response is returned.